COMPUTER NETWORKS — IMPORTANT QUESTIONS & ANSWERS

> **For:** College Exams | Placement Preparation | Technical Interviews | Quick Revision

---

SECTION 1 — NETWORKING FUNDAMENTALS

Q1. What is a computer network? Why is it needed?

Definition

A **computer network** is a collection of **two or more interconnected devices** that can communicate and share resources (data, hardware, services) with each other.

Why It Is Needed

Need	Explanation
Resource Sharing	Share printers, files, storage, internet connection
Communication	Email, messaging, video calls, collaboration
Data Centralization	Store data on servers; access from anywhere
Cost Reduction	Share expensive hardware among multiple users
Reliability	Data can be replicated; backup over network
Remote Access	Access systems and data from anywhere

Types of Networks (by coverage)

Type	Coverage	Example
PAN	Personal (~10m)	Bluetooth, USB
LAN	Building/Campus	Office Wi-Fi, Ethernet
MAN	City	Cable TV, Metro Ethernet
WAN	Country/World	Internet, leased lines

In One Line

> **A computer network connects devices to share data and resources; it is needed for communication, sharing, and centralized management.**

Q2. What is the OSI model? Explain all 7 layers and their functions.

Definition

The **OSI (Open Systems Interconnection) Model** is a **conceptual 7-layer framework** developed by ISO that standardizes how different systems communicate over a network.

The 7 Layers

```
Layer 7 — Application    ← User interacts here (HTTP, FTP, DNS)
Layer 6 — Presentation   ← Format, encrypt, compress
Layer 5 — Session        ← Manage sessions between apps
Layer 4 — Transport      ← End-to-end delivery (TCP, UDP)
Layer 3 — Network        ← Logical addressing, routing (IP)
Layer 2 — Data Link      ← Frame delivery, MAC, error detect
Layer 1 — Physical       ← Raw bits over cables/radio
```

> **Mnemonic (top → bottom):** **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing

Layer-by-Layer Functions

Layer	Name	PDU	Protocols/Devices	Function
7	**Application**	Data	HTTP, FTP, SMTP, DNS	Provides interface for user applications
6	**Presentation**	Data	SSL/TLS, JPEG, ASCII	Translates, encrypts, compresses data
5	**Session**	Data	NetBIOS, RPC	Establishes, manages, terminates sessions
4	**Transport**	Segment	TCP, UDP	End-to-end delivery, reliability, ports
3	**Network**	Packet	IP, ICMP, OSPF, Router	Logical addressing and routing
2	**Data Link**	Frame	Ethernet, ARP, Switch	Physical (MAC) addressing, framing, error detection

1	**Physical**	Bit	Cables, Hub, NIC, Wi-Fi	Transmits raw bits over physical medium

Key Points Per Layer

- **Layer 7 (Application)** → What the user sees (web page, email, DNS query).
- **Layer 6 (Presentation)** → Converts data format (e.g., encrypt for HTTPS, compress JPEG).
- **Layer 5 (Session)** → Opens and manages sessions; handles login/logout.
- **Layer 4 (Transport)** → Segments data; ensures reliable delivery; multiplexes via ports.
- **Layer 3 (Network)** → Adds source/destination IP; routes across networks.
- **Layer 2 (Data Link)** → Adds source/destination MAC; detects errors with CRC.
- **Layer 1 (Physical)** → Converts bits to electrical signals, light, or radio waves.

In One Line

> **OSI is a 7-layer conceptual model; each layer has a defined role — from raw bits (Physical) to user applications (Application).**

Q3. What is the TCP/IP model? How is it different from the OSI model?

Definition

The **TCP/IP Model** (Internet Model) is the **practical 4-layer model** that actually powers the Internet, based on the TCP and IP protocols.

The 4 Layers

```
Layer 4 — Application    (HTTP, FTP, DNS, SMTP, SSH, DHCP)
Layer 3 — Transport      (TCP, UDP)
```

```
Layer 2 — Internet       (IP, ICMP, ARP)
Layer 1 — Network Access (Ethernet, Wi-Fi, physical hardware)
```

OSI vs TCP/IP

Feature	OSI Model	TCP/IP Model
Layers	7	4
Purpose	Conceptual reference	Practical implementation
Usage	Teaching and troubleshooting	Real Internet
Application layer	3 layers (Session + Presentation + Application)	1 combined Application layer
Bottom layers	2 layers (Physical + Data Link)	1 combined Network Access layer
Protocol independence	Protocol-neutral	Built around TCP/IP
Developed by	ISO	ARPANET / DoD

Layer Mapping

```
OSI                    TCP/IP
Application     ■
Presentation    ■■■■■■■■→ Application
Session         ■
Transport       ■■■■■■■■→ Transport
Network         ■■■■■■■■→ Internet
Data Link       ■
```

```
Physical        ■■■■■■■■→ Network Access
```

In One Line

> **TCP/IP is the 4-layer practical model used by the Internet; OSI is the 7-layer conceptual reference model — TCP/IP collapses OSI's top 3 and bottom 2 layers.**

Q4. What is encapsulation and decapsulation in networking?

Encapsulation

Adding headers (and sometimes trailers) at each layer as data travels **down** the sender's protocol stack before transmission.

```
Application:   [Data]
                   ↓  Transport layer adds TCP/UDP header
Transport:     [TCP Header | Data]                    → Segment
                   ↓  Network layer adds IP header
Network:       [IP Header | TCP Header | Data]        → Packet
                   ↓  Data Link adds frame header + FCS trailer
Data Link:     [Frame Header | Packet | FCS]          → Frame
                   ↓  Physical converts to bits
Physical:      01010101... (Bits on the wire)
```

Decapsulation

Removing headers at each layer as data travels **up** the receiver's protocol stack — reverse of encapsulation.

```
Physical:   Bits received on wire
Data Link:  Strips Frame Header + FCS → extracts Packet
Network:    Strips IP Header → extracts Segment
Transport:  Strips TCP/UDP Header → extracts Data
Application: Data reaches the application
```

PDU Names at Each Layer

Layer	PDU Name
Application	Data / Message
Transport	Segment (TCP) / Datagram (UDP)
Network	Packet
Data Link	Frame
Physical	Bit

In One Line

> **Encapsulation adds headers at each layer going down (sender); decapsulation removes headers going up (receiver).**

SECTION 2 — MAC, IP & NETWORK DEVICES

Q5. What is the difference between a MAC address and an IP address?

MAC Address

- **Media Access Control** address.
- **48-bit hardware address** permanently burned into NIC by manufacturer.
- Format: `AA:BB:CC:DD:EE:FF` (hexadecimal).
- Works at **Layer 2 (Data Link)**.
- Used for **local network delivery** (within the same LAN).

IP Address

- **Internet Protocol** address.
- **32-bit (IPv4) or 128-bit (IPv6) logical address** assigned by a network admin or DHCP.
- Used for **routing across networks** globally.
- Works at **Layer 3 (Network)**.

Comparison

Feature	MAC Address	IP Address
Layer	Data Link (Layer 2)	Network (Layer 3)
Type	Hardware (physical)	Logical (software)
Format	48-bit hex (6 bytes)	32-bit (IPv4) or 128-bit (IPv6)
Scope	Local network (LAN)	Global (Internet)
Changes	Fixed (burned-in)	Can change (DHCP)
Assigned by	Manufacturer	Admin / DHCP
Used by	Switch (local delivery)	Router (routing)

In One Line

> **MAC = hardware Layer 2 address for local delivery; IP = logical Layer 3 address for global routing.**

Q6. What is ARP? How does ARP work?

Definition

ARP (Address Resolution Protocol) is a protocol that **maps an IP address to a MAC address** within a local network so that frames can be delivered correctly.

Why ARP is Needed

- A switch delivers frames using **MAC addresses**, not IP addresses.
- When a device knows the destination **IP** but not the **MAC**, it uses ARP to find it.

How ARP Works

```
1. Host A wants to send data to IP: 192.168.1.5
2. A checks its ARP cache → IP not found
3. A sends ARP Request (BROADCAST):
  "Who has 192.168.1.5? Tell 192.168.1.10"
  (sent to FF:FF:FF:FF:FF:FF — all devices receive it)
4. Host B (192.168.1.5) recognizes its own IP
5. B sends ARP Reply (UNICAST) to A:
  "192.168.1.5 is at AA:BB:CC:DD:EE:FF"
6. A stores this in ARP cache and sends the frame
```

Key Terms

Term	Description
ARP Cache	Local table storing IP→MAC mappings (expires after timeout)
ARP Request	Broadcast message asking for MAC of a given IP
ARP Reply	Unicast response with the MAC address
Gratuitous ARP	A host announces its own IP→MAC mapping (used after IP change)

In One Line

> **ARP broadcasts "Who has this IP?" and gets back the MAC address — enabling frame delivery within a LAN.**

Q7. What is the difference between a hub, switch, and router?

Hub

- **Layer 1 (Physical)** device.
- **Broadcasts** all incoming data to **every port** regardless of destination.
- All devices share one **collision domain**.
- **Dumb device** — no intelligence, no MAC table.
- Half-duplex. Largely obsolete.

Switch

- **Layer 2 (Data Link)** device.
- **Forwards frames only to the correct port** using a **MAC address table**.
- Each port = separate **collision domain**; full-duplex.
- **Smart device** — learns MAC addresses dynamically.

Router

- **Layer 3 (Network)** device.
- **Routes packets between different networks** using **IP addresses** and a **routing table**.
- Each interface = separate **broadcast domain**.
- Performs NAT, DHCP, firewall functions.

Comparison Table

Feature	Hub	Switch	Router

OSI Layer	Layer 1	Layer 2	Layer 3
Address used	None	MAC address	IP address
Forwarding method	Broadcast to all	Specific port (MAC table)	Best path (routing table)
Collision domain	1 for all ports	1 per port	1 per interface
Broadcast domain	1	1 (unless VLANs)	Separate per interface
Intelligence	None	MAC learning	Routing protocols
Connects	Devices in same LAN	Devices in same LAN	Different networks

In One Line

> **Hub=broadcasts to all (Layer 1); Switch=smart frame delivery by MAC (Layer 2); Router=routes between networks by IP (Layer 3).**

Q8. What is an IP address? What is the difference between public and private IP addresses?

Definition

An **IP (Internet Protocol) address** is a **unique numerical label** assigned to every device on a network, used for identification and routing.

IPv4 Format

```
192 . 168 . 1 . 1
↑     ↑     ↑   ↑
8 bits  8 bits  8 bits  8 bits  = 32 bits
```

```
Range: 0–255 per octet
```

Public vs Private IP

Feature	Public IP	Private IP
Assigned by	ISP	Router (DHCP) / Admin
Scope	Globally unique and routable	Local network only; not routable on Internet
Cost	Paid (limited supply)	Free (reused everywhere)
Visible to Internet	Yes	No (hidden behind NAT)
Example	8.8.8.8	192.168.1.10

Private IP Ranges (RFC 1918)

Class	Private Range
A	10.0.0.0 – 10.255.255.255
B	172.16.0.0 – 172.31.255.255
C	192.168.0.0 – 192.168.255.255

Special Addresses

- **127.0.0.1** → Loopback (localhost) — self-testing.
- **0.0.0.0** → Any address (default route or unspecified).
- **255.255.255.255** → Limited broadcast.

In One Line

> **IP addresses identify devices on a network; public IPs are globally unique; private IPs are local and translated to public via NAT.**

Q9. What is subnetting? Why is subnetting needed?

Definition

Subnetting is the process of **dividing a large network into smaller sub-networks (subnets)** by borrowing bits from the host portion of an IP address.

Why Subnetting is Needed

Reason	Explanation
Reduce broadcast traffic	Smaller broadcast domain = fewer unnecessary broadcasts
Improve security	Isolate departments; control traffic between subnets
Efficient IP use	Allocate only the addresses needed per segment
Better management	Organize network logically (e.g., per floor, department)
Performance	Fewer devices per subnet = less congestion

Key Terms

- **Subnet Mask** → Defines which bits are network vs host.
- **CIDR notation** → `/24` means 24 bits for network, 8 for hosts.
- **Network address** → First address in subnet (not assignable).

- **Broadcast address** → Last address in subnet (not assignable).
- **Usable hosts** → Total addresses – 2.

Formulas

```
Number of subnets   = 2^(borrowed bits)
Hosts per subnet    = 2^(host bits remaining) – 2
Block size          = 256 – last octet of subnet mask
```

In One Line

> **Subnetting divides a network into smaller parts to reduce broadcasts, improve security, and use IP addresses efficiently.**

Q10. Given an IP address and subnet mask, how do you find the network address, broadcast address, and usable host range?

Method

Given: IP = `192.168.1.130`, Subnet Mask = `255.255.255.192` (/26)

Step 1 — Find block size

```
Last octet of mask = 192
Block size = 256 – 192 = 64
```

Step 2 — Find the subnet

Block boundaries: 0, 64, **128**, 192, 256
IP = .130 → falls in block starting at **128**

Step 3 — Find addresses

```
Network Address  = 192.168.1.128   (first address – not assignable)
Broadcast Address= 192.168.1.191   (next block start – 1 = 128 + 64 – 1)
Usable Host Range= 192.168.1.129 – 192.168.1.190
```

```
Usable Hosts     = 2^6 – 2 = 62
```

General Template

Field	Formula / Rule
Network Address	IP AND Subnet Mask (bitwise)
Broadcast Address	Network Address OR (NOT Subnet Mask)
First Usable Host	Network Address + 1
Last Usable Host	Broadcast Address – 1
Number of Hosts	2^(host bits) – 2

Quick CIDR Reference

CIDR	Mask	Hosts
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62
/27	255.255.255.224	30
/28	255.255.255.240	14
/30	255.255.255.252	2

In One Line

> **Network = first address (IP AND mask); Broadcast = last address; Usable hosts = total − 2 (network + broadcast).**

SECTION 3 — ROUTING & NETWORK COMMUNICATION

Q11. What is routing? How does a router decide where to forward a packet?

Definition

Routing is the process of **selecting the best path** for packets to travel from a source to a destination across multiple networks.

How a Router Decides (Routing Table Lookup)

```
1. Packet arrives at router with destination IP (e.g., 10.0.5.3)
2. Router reads destination IP from packet header
3. Router searches routing table:
   → Performs LONGEST PREFIX MATCH
   → Most specific route wins (e.g., /24 beats /16 beats /0)
4. Router forwards packet out the matching interface to the next-hop IP
5. TTL is decremented by 1; if TTL = 0, packet is dropped (ICMP Time Excee
6. Process repeats at each router until destination reached
```

Routing Table Entry Example

Destination	Mask	Next Hop	Interface	Metric
10.0.5.0	/24	203.0.113.1	eth1	1
192.168.1.0	/24	—	eth0	0

0.0.0.0	/0	203.0.113.254	eth1	10

- **0.0.0.0/0** → Default route (used when no specific match found).
- **Longest Prefix Match** → Most specific route always wins.

Types of Routing

Type	Description
Static Routing	Manually configured by admin; no auto-update
Dynamic Routing	Auto-learned via routing protocols (OSPF, BGP, RIP)
Default Routing	Single default route (0.0.0.0/0) for all unknown destinations

In One Line

> **A router forwards packets using longest prefix match in the routing table, hop by hop, decrementing TTL each time.**

Q12. What is NAT? Why is NAT used?

Definition

NAT (Network Address Translation) is a technique where a **router replaces private IP addresses with a public IP** when forwarding packets to the Internet, and reverses the mapping for incoming replies.

Why NAT is Used

- **IPv4 address exhaustion** → Only ~4.3 billion addresses; far more devices exist.
- **Security** → Internal private IPs are hidden from the Internet.

- **Flexibility** → Internal addressing can change without affecting public IP.

How NAT Works

```
Private Network              Router (NAT)                Internet
192.168.1.10:5000 ■■→  [translate to]  203.0.113.1:5001 ■■→ Web Server
                        [NAT Table]
192.168.1.11:6000 ■■→  [translate to]  203.0.113.1:5002 ■■→ Web Server

Reply from Web Server: 203.0.113.1:5001 ■■→ [reverse translate] ■■→ 192.
```

Types of NAT

Type	Description
Static NAT	One-to-one: one private IP → one public IP
Dynamic NAT	Pool of public IPs; assigned dynamically
PAT / Masquerading (NAPT)	Many private IPs → one public IP; differentiated by port numbers

> **PAT (Port Address Translation)** is the most common type — used in home routers.

In One Line

> **NAT translates private IPs to a public IP for Internet access, solving IPv4 exhaustion and hiding internal addresses.**

Q13. What is the difference between a packet, segment, frame, and data?

The PDU Hierarchy

```
Application Layer:  DATA / MESSAGE
                        ↓ Transport layer adds header
Transport Layer:    SEGMENT (TCP) / DATAGRAM (UDP)
                        ↓ Network layer adds IP header
Network Layer:      PACKET
                        ↓ Data Link layer adds frame header + FCS trailer
Data Link Layer:    FRAME
                        ↓ Physical layer converts to signals
Physical Layer:     BITS
```

Comparison

Term	Layer	Contains	Key Info
Data/Message	Application	Raw user data	HTTP body, email content
Segment	Transport (Layer 4)	Data + TCP/UDP header	Source/destination port, seq no., ACK
Packet	Network (Layer 3)	Segment + IP header	Source/destination IP address, TTL
Frame	Data Link (Layer 2)	Packet + MAC header + FCS	Source/destination MAC, CRC
Bit	Physical (Layer 1)	Electrical/light signals	Raw 0s and 1s

Key Rule

- Each layer **wraps the previous layer's PDU** by adding its own header (encapsulation).
- Each layer **unwraps** on the receiver side (decapsulation).

In One Line

> **Data → Segment (+ ports) → Packet (+ IP) → Frame (+ MAC + FCS) → Bits — each layer adds addressing for its scope.**

SECTION 4 — TCP & UDP

Q14. What is the difference between TCP and UDP? When would you use each?

TCP — Transmission Control Protocol

- **Connection-oriented** → 3-way handshake before data transfer.
- **Reliable** → ACKs + retransmission of lost packets.
- **Ordered** → Sequence numbers ensure correct order.
- **Flow control** → Sliding window (receiver controls rate).
- **Congestion control** → Slow start, congestion avoidance.
- **Overhead** → 20-byte header.

UDP — User Datagram Protocol

- **Connectionless** → No handshake; just send.
- **Unreliable** → No ACKs; no retransmission.
- **No ordering** → Packets may arrive out of order.
- **No flow/congestion control**.
- **Low overhead** → 8-byte header; very fast.

Comparison Table

Feature	TCP	UDP

Connection	Connection-oriented	Connectionless
Reliability	Reliable (ACK + retransmit)	Unreliable
Ordering	Ordered (sequence numbers)	Unordered
Speed	Slower	Faster
Header size	20–60 bytes	8 bytes
Flow control	Yes	No
Congestion control	Yes	No
Use cases	HTTP/S, FTP, SMTP, SSH, Telnet	DNS, DHCP, VoIP, video streaming, online gaming

When to Use Each

- **Use TCP** → When data integrity and order are critical (file download, email, banking).
- **Use UDP** → When speed is critical and some loss is acceptable (live video, VoIP, gaming, DNS lookups).

TCP vs UDP — What Happens on Packet Loss

```
TCP: Detects loss (timeout/dup ACK) → retransmits lost packet → receiver
UDP: Lost packet is dropped → receiver gets whatever arrived → no retrans
    (packets 1, 2, [X], 4, 5 → receiver gets 1, 2, 4, 5 — packet 3 lost fo
```

In One Line

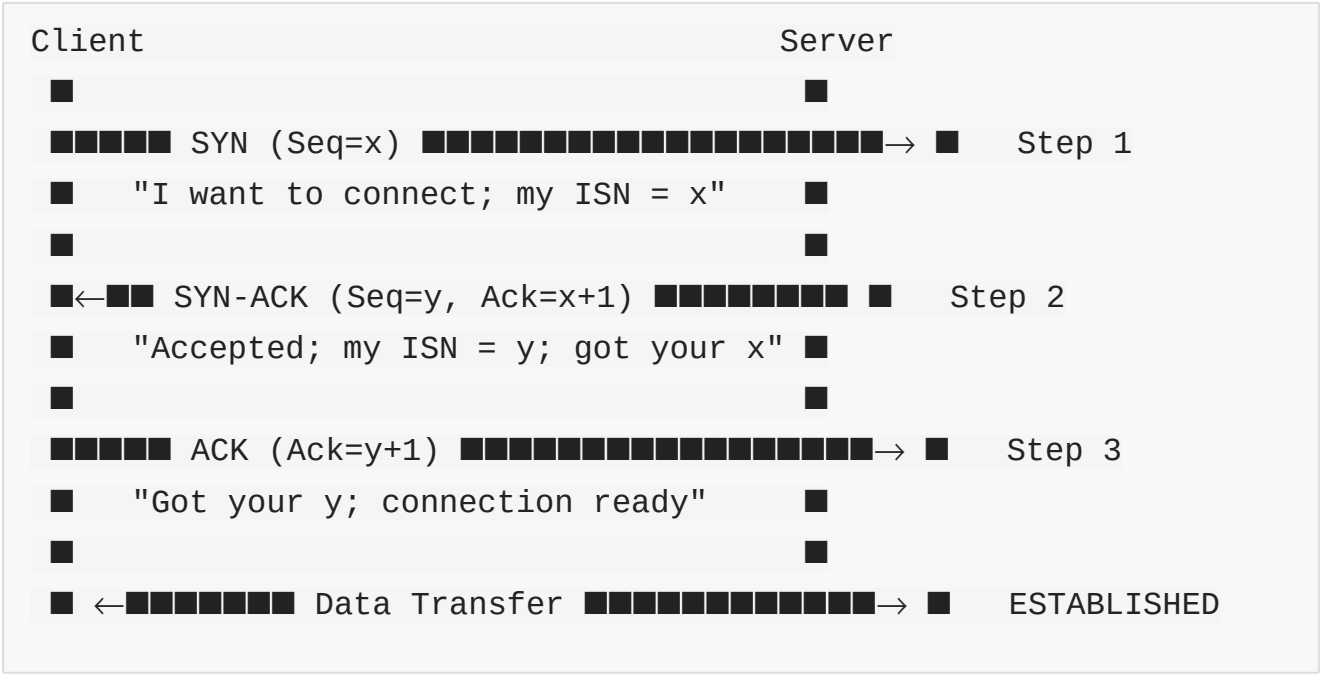
> **TCP is reliable, ordered, slow; UDP is fast, unreliable, unordered — use TCP for accuracy, UDP for speed.**

Q15. What is the TCP 3-way handshake? Why are SYN, SYN-ACK, and ACK required?

Definition

The **TCP 3-Way Handshake** is the process TCP uses to **establish a reliable connection** between a client and server before any data is exchanged.

The 3 Steps



Why Each Step is Needed

Step	Flag	Direction	Purpose
1	**SYN**	Client → Server	Client announces intent to connect and shares its Initial Sequence Number (ISN)

2	**SYN-ACK**	Server → Client	Server accepts connection, acknowledges client's ISN, shares its own ISN
3	**ACK**	Client → Server	Client acknowledges server's ISN; both sides are synchronized and ready

Why 3 Steps and Not 2?

- **2 steps are not enough** → Server needs to know that the client received the server's ISN.
- Both sides need to **confirm they can send AND receive**.
- The 3rd ACK confirms the client received the server's SYN-ACK → bidirectional channel verified.

In One Line

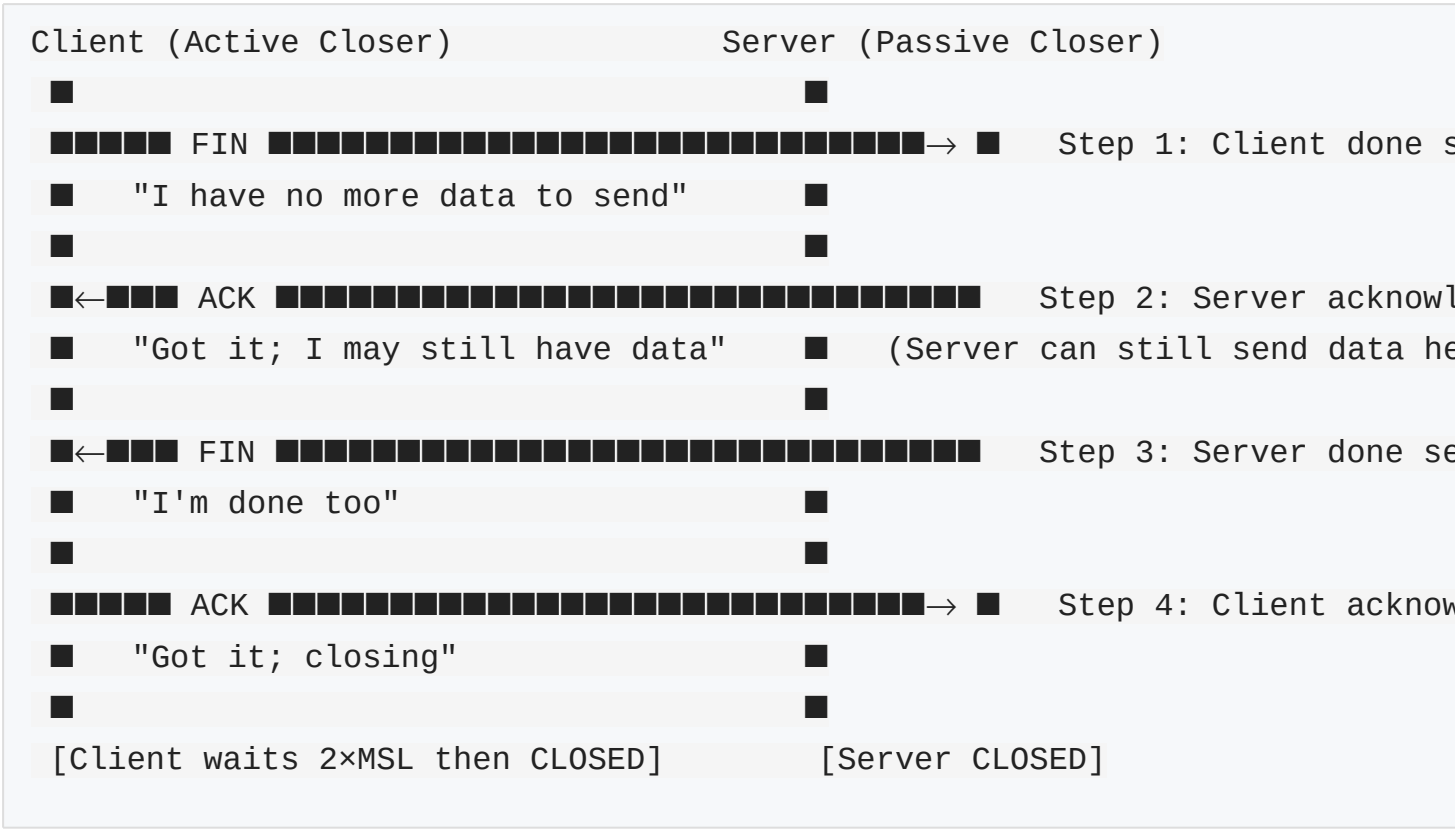
> **3-Way Handshake (SYN → SYN-ACK → ACK) synchronizes sequence numbers and confirms both sides can send and receive before data exchange.**

Q16. How does TCP terminate a connection? Explain the 4-way handshake.

Definition

TCP 4-Way Termination is the graceful process of **closing a TCP connection**, where each side independently closes its half of the connection.

The 4 Steps



Why 4 Steps and Not 3?

- Each direction of the connection is **closed independently**.
- After receiving the client's FIN (Step 1), the **server may still have data to send**.
- So Steps 2 (ACK) and 3 (FIN) are separate — the server's ACK and FIN are not simultaneous.

TIME_WAIT State

- Client enters **TIME_WAIT** after sending final ACK.
- Lasts **2 × MSL (Maximum Segment Lifetime)** ≈ 60–120 seconds.
- Ensures final ACK reached server; old duplicate packets expire.

In One Line

> **TCP 4-way termination (FIN → ACK → FIN → ACK) closes each direction independently; TIME_WAIT ensures clean closure.**

Q17. How does TCP provide reliable data transmission?

TCP Reliability Mechanisms

Mechanism	How It Helps
Sequence Numbers	Each byte numbered; receiver reorders segments correctly
Acknowledgements (ACK)	Receiver confirms received bytes; sender knows what arrived
Retransmission	If ACK not received within timeout → sender retransmits
Checksums	Detects bit errors in segments; corrupted segments discarded
Duplicate ACKs / Fast Retransmit	3 duplicate ACKs → retransmit immediately without waiting for timeout
Connection Setup	3-way handshake ensures both sides are ready before data
Ordered Delivery	Sequence numbers allow reassembly in correct order
Flow Control	Prevents receiver buffer overflow (receive window)
Congestion Control	Prevents network overload (slow start, congestion avoidance)

Reliable Delivery Process

```
Sender: sends Seg[1], Seg[2], Seg[3]
        ■■ starts timer for each

Receiver: gets Seg[1], Seg[2] → sends ACK 3 (next expected)
         Seg[3] lost → timer expires → Sender retransmits Seg[3]
         Or: Receiver sends 3× ACK 3 → Fast Retransmit (no timeout wait)
```

In One Line

> **TCP ensures reliability via sequence numbers, ACKs, retransmission (timeout + fast retransmit), checksums, and ordered delivery.**

Q18. What is TCP flow control? How does the sliding-window mechanism work?

Definition

TCP Flow Control is a mechanism that prevents the **sender from overwhelming the receiver's buffer** by limiting how much unacknowledged data can be in transit.

Receive Window (rwnd)

- Receiver advertises its **available buffer space** as `rwnd` in every ACK.
- Sender limits unacknowledged data to ≤ `rwnd`.
- If `rwnd = 0` → Sender pauses; sends 1-byte probe segments to check when receiver is ready.

Sliding Window Mechanism

```
Window size = 4 segments

Sequence:  1  2  3  4  5  6  7  8  9  10
          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Sent+ACKed ■ 1  2 ■ Sent, not ACKed ■ 3  4  5  6 ■ Can send ■ 7  8 ■ Futu
          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                   ■■■■ Window (size 4) ■■■■
```

```
→ ACK 3 received → window slides right:
          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Sent+ACKed ■ 1  2  3 ■ Not ACKed ■ 4  5  6  7 ■ Can send ■ 8  9 ■
          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Key Points

- Window "slides" forward as ACKs are received.
- **Larger window** → More data in flight → Higher throughput.
- **Smaller window** → Less data in flight → Lower throughput, more predictable.
- **Go-Back-N** → On error, retransmit from errored packet onwards.
- **Selective Repeat** → Retransmit only the specific lost packet.

In One Line

> **Flow control uses the receiver's advertised window (rwnd) to limit sender speed; sliding window keeps multiple packets in flight for efficiency.**

Q19. What is TCP congestion control? Explain slow start, congestion avoidance, and retransmission.

Definition

TCP Congestion Control dynamically adjusts the **sending rate** to prevent overwhelming the **network** (routers, links), not just the receiver.

Congestion Window (cwnd)

- Sender maintains a **congestion window (cwnd)** — limits unACKed data in the network.
- Actual send rate = min(cwnd, rwnd).

Phases of TCP Congestion Control

1. Slow Start

```
Start: cwnd = 1 MSS (Maximum Segment Size)
Every RTT: cwnd doubles (exponential growth)
          cwnd = 1 → 2 → 4 → 8 → 16 ...
Until: cwnd reaches ssthresh (slow start threshold)
```

2. Congestion Avoidance

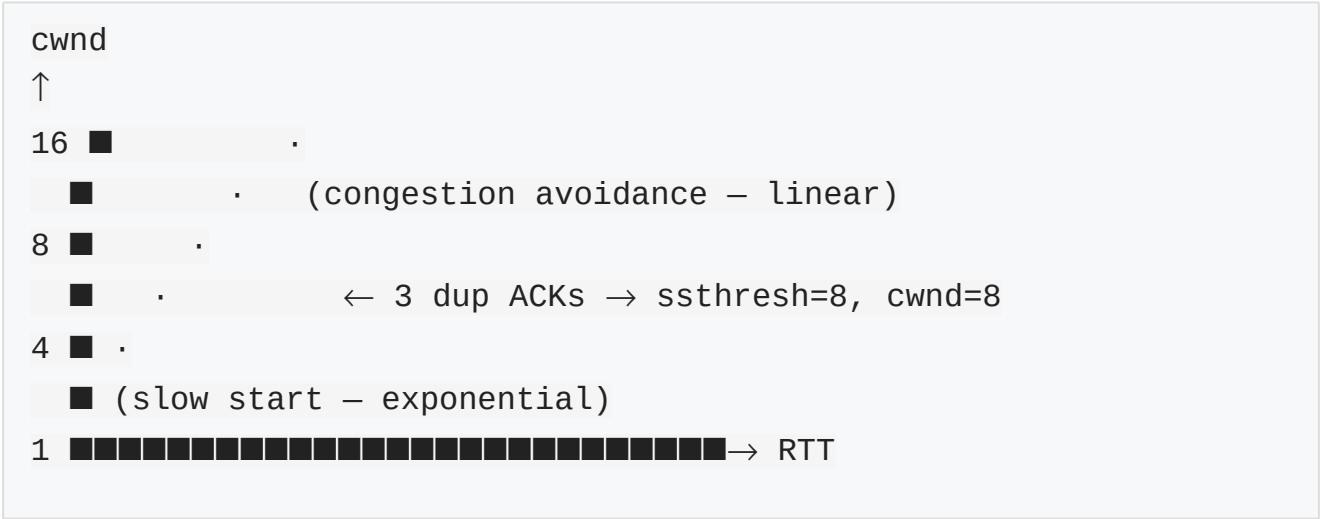
```
After ssthresh is reached:
cwnd increases by 1 MSS per RTT (linear/additive growth)
cwnd = 16 → 17 → 18 → 19 ...
Until: packet loss detected
```

3. On Packet Loss (Congestion Detected)

```
Case A – Timeout (severe congestion):
   ssthresh = cwnd / 2
   cwnd = 1 MSS
   Restart Slow Start from cwnd = 1

Case B – 3 Duplicate ACKs (moderate congestion – Fast Recovery):
   ssthresh = cwnd / 2
   cwnd = ssthresh    (skip slow start – go directly to congestion avoidan
```

Visual Summary



In One Line

> **TCP congestion control: Slow Start (exponential growth) → Congestion Avoidance (linear) → packet loss → reduce cwnd → repeat.**

SECTION 5 — APPLICATION & INTERNET PROTOCOLS

Q20. What happens when you type google.com in a browser and press Enter?

Complete Step-by-Step

```
1. URL Parsing
   Browser parses: protocol=HTTPS, domain=google.com, path=/

2. DNS Resolution
   a. Browser checks its DNS cache → not found
   b. Checks OS DNS cache → not found
   c. Queries Recursive DNS Resolver (ISP's DNS)
   d. Resolver queries Root DNS Server → "ask .com TLD"
   e. Resolver queries .com TLD Server → "ask google.com's NS"
   f. Resolver queries Google's Authoritative DNS → gets IP (142.250.x.x)
   g. IP returned to browser; cached

3. TCP Connection (3-Way Handshake)
   Browser connects to 142.250.x.x port 443 (HTTPS)
   SYN → SYN-ACK → ACK

4. TLS Handshake (for HTTPS)
   Browser and server negotiate cipher suite
   Server sends SSL certificate → browser verifies with CA
   Session key established → all data encrypted

5. HTTP Request
```

```
  Browser sends: GET / HTTP/1.1  Host: google.com

6. Server Processes Request
  Google's server processes the GET request
  Generates HTML response

7. HTTP Response
  Server sends: HTTP/1.1 200 OK + HTML content

8. Browser Renders Page
  Browser parses HTML, CSS, JS
  Additional requests for images, scripts, fonts
  Page displayed to user

9. TCP Connection Closed (4-way termination)
  FIN → ACK → FIN → ACK
```

In One Line

> **URL parse → DNS resolve → TCP connect → TLS handshake → HTTP request → server response → browser render.**

Q21. What is DNS? Explain how DNS resolution works.

Definition

DNS (Domain Name System) is a **hierarchical, distributed naming system** that translates **human-readable domain names** (google.com) into **IP addresses** (142.250.x.x) that computers use to communicate.

DNS Hierarchy

```
Root DNS Servers (13 sets globally — .)
   ↓
TLD DNS Servers (.com, .org, .in, .net)
   ↓
```

```
Authoritative DNS Servers (google.com, amazon.com)
```

DNS Resolution Process (Iterative)

```
Browser → "What is the IP of www.google.com?"
   ↓
1. Local DNS cache check → MISS
2. OS DNS cache check → MISS
3. → Recursive Resolver (ISP's DNS Server)
4. → Root Server: "I don't know google.com, ask .com TLD"
5. → .com TLD Server: "Ask Google's authoritative NS"
6. → Google's Authoritative NS: "www.google.com = 142.250.x.x"
7. → Resolver caches + returns IP to browser
```

DNS Record Types

Record	Purpose	Example
A	Domain → IPv4	google.com → 142.250.1.1
AAAA	Domain → IPv6	google.com → 2607:f8b0::...
CNAME	Alias → canonical name	www → google.com
MX	Mail server for domain	gmail.com → mail.google.com
NS	Authoritative name servers	google.com NS → ns1.google.com
PTR	Reverse DNS (IP → domain)	1.1.250.142 → google.com

TXT	Text data (SPF, DKIM)	"v=spf1 include:_spf.google.com"

Key Terms

- **TTL (Time to Live)** → How long a DNS record is cached before refresh.
- **Recursive Resolver** → Does the full lookup on behalf of the client.
- **Iterative Query** → Resolver asks each server in turn; each responds with a referral.

In One Line

> **DNS translates domain names to IPs; resolution goes local cache → recursive resolver → root → TLD → authoritative server.**

Q22. What is DHCP? Explain the DORA process.

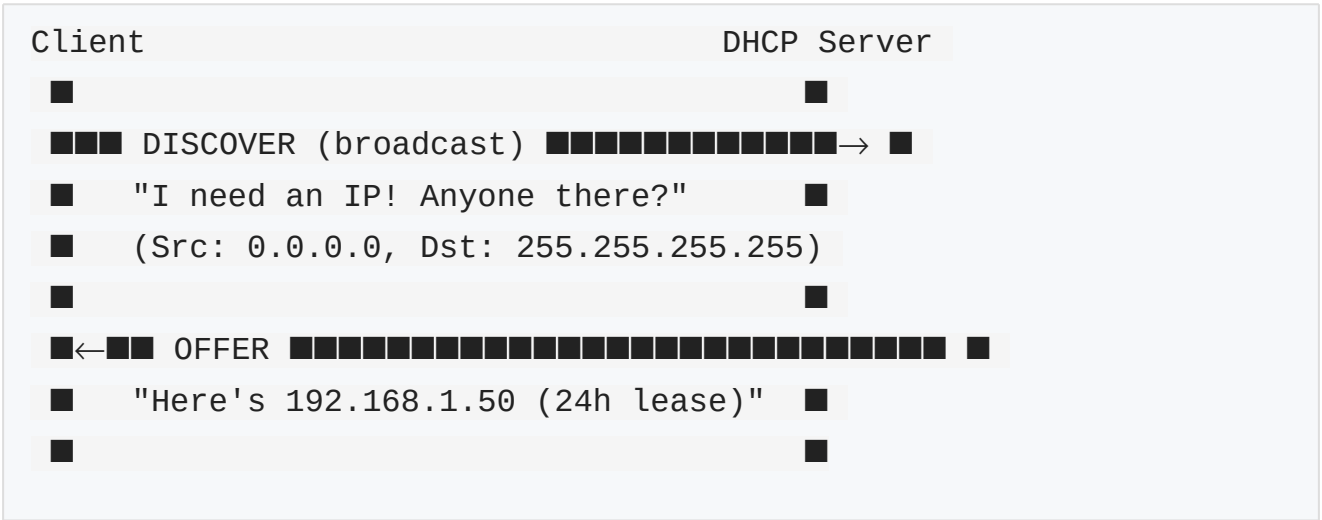
Definition

DHCP (Dynamic Host Configuration Protocol) is a network protocol that **automatically assigns IP addresses and network configuration** to devices when they join a network.

What DHCP Provides

- IP address, Subnet mask, Default gateway, DNS server IPs, Lease duration.

DORA Process



```
■■■ REQUEST (broadcast) ■■■■■■■■■■■■■→ ■
■   "I'll take 192.168.1.50, thanks"   ■
■                                      ■
■←■■ ACKNOWLEDGE ■■■■■■■■■■■■■■■■■■■■■ ■
■   "It's yours. Lease: 24 hours"      ■
■                                      ■
[Client configures interface with assigned IP]
```

Step	Name	Direction	Purpose
D	**Discover**	Client → Broadcast	Client looks for DHCP server
O	**Offer**	Server → Client	Server offers an IP address
R	**Request**	Client → Broadcast	Client requests the offered IP
A	**Acknowledge**	Server → Client	Server confirms the assignment

Key Points

- Lease expires → client must **renew** (sends REQUEST before lease ends).
- Multiple DHCP servers can exist → client picks first offer.
- DHCP uses **UDP port 67** (server) and **68** (client).

In One Line

> **DHCP auto-assigns IP config via DORA (Discover → Offer → Request → Acknowledge) over UDP ports 67/68.**

Q23. What is HTTP? Explain HTTP methods and common HTTP status codes.

Definition

HTTP (HyperText Transfer Protocol) is an **application-layer, stateless, request-response protocol** used to transfer web pages and data between a client (browser) and a web server.

Key Characteristics

- **Stateless** → Server remembers nothing between requests (cookies/sessions add state).
- **Port 80** (HTTP) / **Port 443** (HTTPS).
- Uses **TCP** as transport.
- Text-based headers; binary or text body.

HTTP Request Structure

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
User-Agent: Mozilla/5.0
Authorization: Bearer <token>
```

HTTP Methods

Method	Purpose	Has Body?	Idempotent?	Safe?
GET	Read/retrieve a resource	No	Yes	Yes
POST	Create a new resource	Yes	No	No

PUT	Replace entire resource	Yes	Yes	No
PATCH	Partially update a resource	Yes	No	No
DELETE	Delete a resource	Optional	Yes	No
HEAD	Like GET but no response body	No	Yes	Yes
OPTIONS	List supported HTTP methods	No	Yes	Yes

- **Safe** → Doesn't modify server state (GET, HEAD).
- **Idempotent** → Multiple identical calls = same result (GET, PUT, DELETE).

HTTP Status Codes

Range	Category	Common Examples
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content

3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

HTTP Response Structure

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024

<html>...</html>
```

In One Line

> **HTTP is a stateless request-response protocol; GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove; 2xx=OK, 4xx=client error, 5xx=server error.**

Q24. What is the difference between HTTP and HTTPS? How does TLS/SSL secure communication?

HTTP vs HTTPS

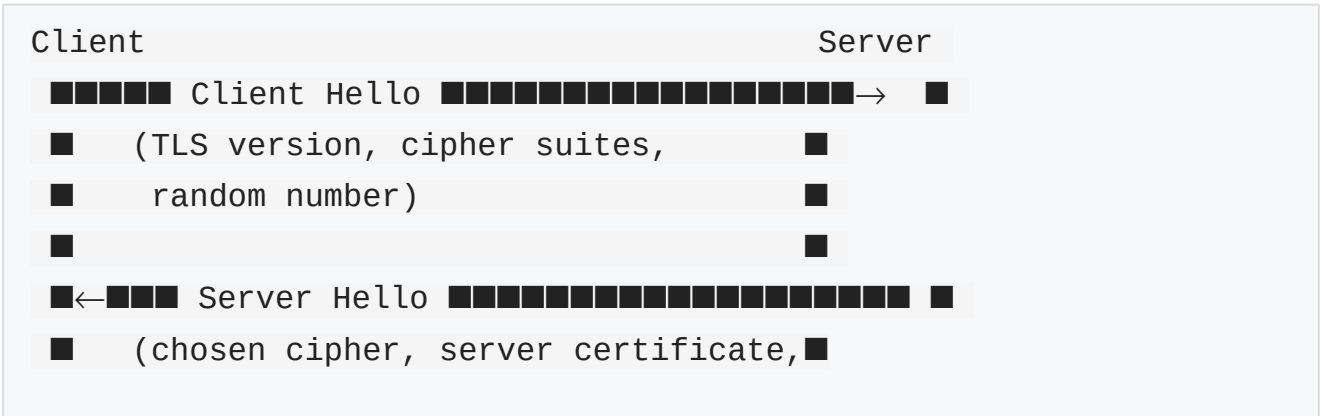
Feature	HTTP	HTTPS

Full form	HyperText Transfer Protocol	HTTP Secure
Port	80	443
Encryption	No (plaintext)	Yes (TLS)
Data visible to ISP/attacker	Yes (MITM possible)	No (encrypted)
Authentication	No server verification	Server verified via digital certificate
Certificate required	No	Yes (from CA)
URL prefix	`http://`	`https://`
Speed	Slightly faster	Slightly slower (TLS overhead)

How TLS/SSL Secures Communication

TLS (Transport Layer Security) provides three guarantees: 1. **Confidentiality** → Data is encrypted; cannot be read if intercepted. 2. **Integrity** → Data cannot be tampered without detection. 3. **Authentication** → Server's identity is verified via digital certificate.

TLS Handshake (Simplified)



```
■    server random)                    ■
■                                     ■
■ Client verifies certificate with CA  ■
■ (Is this really google.com?)         ■
■                                     ■
■■■■■ Key Exchange ■■■■■■■■■■■■■■■■■→  ■
■   (client sends pre-master secret,   ■
■    encrypted with server's public key■
■                                     ■
[Both sides derive same session key from shared secrets]
■                                     ■
■←■■■■■■■■ Encrypted Data ■■■■■■■■■■■→ ■
■   (all HTTP data encrypted with      ■
■    symmetric session key — AES)      ■
```

Why Asymmetric + Symmetric?

- **Asymmetric (RSA/ECDH)** → Used during handshake to securely exchange session key (slow).
- **Symmetric (AES)** → Used for actual data encryption (fast).

In One Line

> **HTTPS = HTTP + TLS; TLS uses asymmetric crypto for handshake/key exchange, then symmetric crypto for fast encrypted data transfer.**

Q25. What is a port and what is a socket? How does a client communicate with a server using sockets?

Port

- A **port** is a **16-bit logical number (0–65535)** that identifies a specific **application or service** on a host.
- IP address identifies the **host**; port identifies the **service** on that host.

Socket

- A **socket** is an **endpoint for network communication**, uniquely identified by:

```
Socket = IP Address : Port : Protocol
Example: 192.168.1.10 : 5000 : TCP
```

- A **connection** is identified by a **pair of sockets**:
- Client socket: `ClientIP:ClientPort`
- Server socket: `ServerIP:ServerPort`

Client-Server Communication Using Sockets

```
SERVER SIDE:                              CLIENT SIDE:

1. socket()   → create socket             1. socket() → create socket
2. bind()     → bind to IP:Port
3. listen()   → mark as passive           2. connect() → 3-way handshake ■■
4. accept()   ← waits for connection      3. send()    → send request data
5. recv()     → read client request       4. recv()    ← receive response
6. send()     → send response             5. close()   → close connection
7. close()    → close connection
```

Socket Communication Flow

```
Client                                        Server
 ■■■■■ connect (SYN) ■■■■■■■■■■■■■■■■■■■→ ■  ← listen() + accept()
 ■←■■■ SYN-ACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 ■■■■■ ACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■→ ■  ← Connection ESTABLIS
 ■                                          ■
 ■■■■■ send("GET / HTTP/1.1\r\n") ■■■■→ ■  ← recv()
 ■←■■■ recv("HTTP/1.1 200 OK\r\n") ■■■ ■  ← send()
 ■                                          ■
 ■■■■■ close (FIN) ■■■■■■■■■■■■■■■■■■■→ ■
 ■←■■■ ACK → FIN ■■■■■■■■■■■■■■■■■■■■■■■■
 ■■■■■ ACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■→ ■  ← Connection CLOSED
```

Well-Known Port Numbers

Port	Service
22	SSH
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL

In One Line

> **Port identifies a service on a host; Socket = IP + Port + Protocol; client uses connect() to handshake with server's listen()/accept() and then exchanges data via send()/recv().**

QUICK REVISION TABLE

#	Question	Key Answer
1	What is a computer network?	Devices connected to share data/resources

2	What is the OSI model?	7-layer reference: Physical → Data Link → Network → Transport → Session → Presentation → Application
3	What is the TCP/IP model?	4-layer practical model: Network Access → Internet → Transport → Application
4	What is encapsulation?	Add headers at each layer (down); remove headers at each layer (up = decapsulation)
5	MAC vs IP address	MAC=Layer 2, hardware, LAN; IP=Layer 3, logical, global routing
6	How does ARP work?	Broadcast "Who has IP X?" → Target replies with MAC → stored in ARP cache
7	Hub vs Switch vs Router	Hub=broadcast/L1; Switch=MAC table/L2; Router=routing table/L3
8	Public vs Private IP	Public=globally routable/ISP; Private=local only/RFC1918 (10.x, 172.16-31.x, 192.168.x)

9	Why subnetting?	Reduce broadcasts, improve security, efficient IP use
10	Find network/broadcast address	Network=IP AND mask; Broadcast=last IP in block; Hosts=2^h–2
11	How does routing work?	Longest prefix match in routing table → forward to next-hop → TTL decremented
12	What is NAT?	Translates private→public IP; PAT uses ports for many-to-one; solves IPv4 exhaustion
13	Packet vs Segment vs Frame	Data→Segment(+ports)→Packet(+IP)-
14	TCP vs UDP	TCP=reliable,ordered,slow; UDP=fast,unreliable,unordered
15	TCP 3-way handshake	SYN→SYN-ACK→ACK; synchronizes ISNs; verifies bidirectional channel
16	TCP 4-way termination	FIN→ACK→FIN→ACK; each side closes independently; TIME_WAIT = 2×MSL

17	TCP reliability	Seq numbers + ACKs + retransmission + checksums + ordered delivery
18	TCP flow control	rwnd limits sender to receiver's buffer; sliding window keeps multiple packets in flight
19	TCP congestion control	Slow start (exponential) → congestion avoidance (linear) → loss → reduce cwnd
20	What happens on google.com?	URL parse→DNS→TCP connect→TLS handshake→HTTP GET→response→render
21	How DNS works?	Cache→Resolver→Root→TLD→Auth NS→IP returned
22	DHCP DORA process	Discover(broadcast)→Offer→Request
23	HTTP methods and status codes	GET/POST/PUT/PATCH/DELETE; 2xx=OK, 3xx=redirect, 4xx=client, 5xx=server
24	HTTP vs HTTPS / TLS	HTTPS=HTTP+TLS; TLS: asymmetric handshake+session key→symmetric data encryption

25	Port and Socket	Port=service ID; Socket=IP+Port+Protocol; connect()→handshake→send/recv→cl

---